

Отчет по лабораторной работе № 3

Архитектурный проект микросервисной информационной системы LegalAssist AI

Автор: Дементьев Антон Павлович

Поток: ПИС 2.3

Группа: K3341

Ссылка на репозиторий проекта:

https://github.com/AntonDementiev/Information_Systems_Design_ITMO/tree/lab-3

Общая информация

Название работы: Архитектурный проект микросервисной информационной системы автоматизированного юридического помощника LegalAssist AI.

Цель работы: Разработать архитектурный проект микросервисной информационной системы LegalAssist AI по заданной структуре: от контекстной диаграммы и доменного моделирования до инфраструктуры и CI/CD.

Задачи работы:

- Описать архитектурное видение, цели системы и архитектурные драйверы.
- Провести доменное моделирование и выделить bounded contexts.
- Спроектировать состав микросервисов и их взаимодействие.
- Описать архитектуру данных, паттерны согласованности и стратегии миграции.
- Спроектировать внешний API, межсервисные контракты и интеграции.
- Описать кросс-сервисные решения: масштабируемость, надёжность, безопасность, наблюдаемость.
- Описать инфраструктурную архитектуру и CI/CD.

1. Архитектурное видение и контекст

1.1. Описание целей системы и архитектуры

LegalAssist AI — автоматизированный юридический помощник, способный анализировать правовые документы (договоры, иски, доверенности) и предоставлять консультации со ссылками на актуальное законодательство РФ.

Бизнес-цели:

- Снизить стоимость первичной юридической консультации за счёт автоматизации анализа документов и генерации ответов через LLM.
- Обеспечить доступность сервиса 24/7: пользователь получает ответ без участия живого юриста.
- Гарантировать прозрачность: каждый вывод системы подкреплён ссылкой на конкретную норму закона.
- Обеспечить соответствие 152-ФЗ (защита персональных данных) и 63-ФЗ (электронная подпись).

Ключевые сценарии использования:

- Загрузка юридического документа и получение анализа рисков с цитированием норм.
- Диалоговая консультация в чате по конкретному правовому вопросу.
- Поиск по базе знаний (нормативные акты, судебная практика).

Ожидаемые метрики:

- Нагрузка: до 500 одновременных сессий, до 5 000 документов в сутки.
- SLA: 99,5% uptime; RTO < 1 ч; RPO < 15 мин.
- Скорость релизов: двухнедельные спринты, автодеплой в staging, релиз в prod раз в две недели.

1.2. Архитектурные драйверы

Перечень нефункциональных требований (НФТ) с приоритизацией:

- Безопасность [Критический]. Шифрование каналов (TLS 1.3), авторизация через JWT/RBAC, обязательный журнал аудита — критично для юридических данных.
- Надёжность [Высокий]. SLA 99,5%, RTO < 1 ч, RPO < 15 мин; паттерны Circuit Breaker, Retry, Dead Letter Queue.
- Масштабируемость [Высокий]. Горизонтальное масштабирование Analysis Service и Document Service под пиковую нагрузку (LLM-запросы ресурсоёмки).
- Производительность [Средний]. P95 latency API < 300 мс (без времени LLM); LLM-ответ < 30 с.
- Сопровождаемость [Средний]. IaC (Terraform), версионирование API (v1/v2), CI/CD pipeline, единый стек наблюдаемости.
- Интегрируемость [Средний]. REST + Webhook, OpenAPI 3.0, поддержка смены LLM-провайдера через ACL.

1.3. Контекстная диаграмма (уровень System Context)

Внешние акторы и системы:

- Пользователь (физическое лицо или организация) — взаимодействует через Web- или мобильный клиент.
- Администратор — управляет пользователями и базой знаний через Admin Panel.
- LLM Provider (OpenAI / YandexGPT / локальная модель) — внешний сервис генерации ответов.
- Elasticsearch — поисковый движок по нормам права (может быть внешним или внутренним).
- Email/Push Provider (SendGrid, Firebase FCM) — доставка уведомлений пользователям.
- Object Storage (S3 / MinIO) — хранение бинарных файлов загруженных документов.

Основные потоки данных: пользователь загружает документ и отправляет вопрос → система анализирует документ, обращается к LLM и базе знаний → возвращает результат с цитатами из законодательства → Notification Service уведомляет пользователя о готовности ответа.

Рис. 1 — Контекстная диаграмма LegalAssist AI (C4 Level 1)

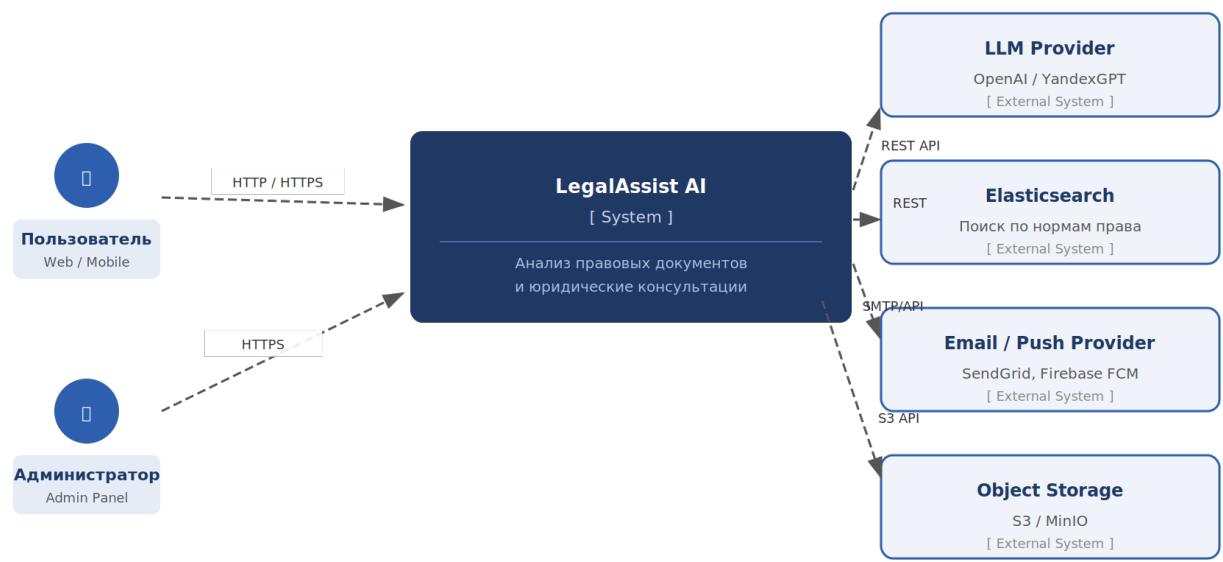


Рис. 1 — Контекстная диаграмма LegalAssist AI (текстовая нотация C4 Level 1)

Микрорефлексия. Раздел помог сформулировать чёткие бизнес-цели и расставить приоритеты НФТ. Для юридической системы безопасность оказывается выше производительности — это стало неожиданным, но логичным выводом: задержка в несколько секунд менее критична, чем утечка персональных данных клиента. Впервые явно фиксировал метрики SLA/RPO на этапе видения — это дисциплинирует всё последующее проектирование.

2. Логическая архитектура и доменное моделирование

2.1. Выделение доменных областей и bounded contexts

На основе анализа предметной области выделены семь доменов:

- Identity (User & Access Management) — регистрация, аутентификация, управление ролями (RBAC).
- Document Management — загрузка, хранение, версионирование и извлечение текста из документов.
- Consultation & Chat — создание консультаций, ведение диалоговых сессий, история сообщений.
- AI Analysis — оркестрация LLM-запросов, формирование результатов анализа, цитирование норм.
- Legal Knowledge Base — хранение и полнотекстовый поиск по нормативным актам и судебной практике.
- Notification — доставка Email/Push-уведомлений пользователям.
- Audit & Compliance — сбор и хранение журнала всех значимых событий системы.

2.2. Доменные модели внутри ключевых контекстов

AI Analysis Context (ключевой):

- Арперат AnalysisJob: id, consultationId, status (PENDING / RUNNING / DONE / FAILED), modelName, startedAt, finishedAt.
- Сущность AnalysisResult: id, jobId, riskLevel (LOW / MEDIUM / HIGH), conclusion (TEXT), confidence (FLOAT).
- Value Object Citation: analysisResultId, legalProvisionId, snippet — неизменяем после создания.
- Инвариант: AnalysisResult может быть создан только для AnalysisJob в статусе DONE.

Document Management Context:

- Арперат Document: id, ownerId, title, docType, status (UPLOADED / PROCESSING / READY / ERROR).
- Сущность DocumentVersion: id, documentId, versionNo, fileUri, checksum, extractedText.
- Инвариант: UNIQUE(documentId, versionNo) — недопустимо создание двух версий с одним номером.

2.3. Межсценарные взаимодействия

Доменные события и команды:

- DocumentUploaded — Document Service публикует при успешной загрузке файла.
- TextExtractionCompleted — Document Service завершил OCR/парсинг.
- ConsultationCreated — Consultation Service создал новую консультацию.
- AnalysisRequested — Consultation Service запросил анализ документа.
- AnalysisCompleted — Analysis Service завершил анализ, результат готов.
- AnalysisFailed — Analysis Service зафиксировал ошибку (LLM недоступен и т.п.).
- UserNotified — Notification Service подтвердил доставку уведомления.

Ключевой бизнес-процесс — «Анализ загруженного документа»:

(1) Пользователь загружает файл → Document Service публикует DocumentUploaded. (2) Text Extraction Worker обрабатывает файл → TextExtractionCompleted. (3) Пользователь создаёт консультацию и прикрепляет документ → ConsultationCreated. (4) Consultation Service публикует AnalysisRequested. (5) Analysis Service получает событие, формирует промпт с извлечённым текстом, вызывает LLM, сохраняет результат → AnalysisCompleted. (6) Notification Service отправляет пользователю уведомление о готовности ответа.

Микрорефлексия. Работа над доменным моделированием показала, что выделение агрегатов сразу определяет границы транзакционной консистентности. Паттерн Aggregate Root я применял впервые осознанно — стало понятно, почему он решает не только задачу группировки сущностей, но и задачу изоляции инвариантов. Доменные события оказались удобным языком для описания бизнес-процессов.

3. Архитектура микросервисов

3.1. Состав микросервисов

Сервис	Домен	Ответственность	Основные операции	Владеемые данные
API Gateway	Инфраструктура	Маршрутизация, аутентификация, rate-limiting	Роутинг HTTP/WS, валидация JWT, throttling	—
User Service	Identity	Управление аккаунтами и ролями	register, login, updateProfile, assignRole	users, roles, user_roles
Document Service	Документы	Приём, хранение и версионирование документов	upload, getVersions, extractText, delete	documents, document_versions
Consultation Service	Консультации	Создание консультаций и ведение диалогов	createConsult, attachDoc, getHistory, addMessage	consultations, chat_sessions, chat_messages
Analysis Service	Анализ / LLM	Оркестрация LLM-запросов, результаты анализа	analyzeConsultation, getResult, retry	analysis_results, citations
Legal KB Service	База знаний	Индексация и поиск норм права	search, getProvision, syncSources	legal_sources, legal_provisions
Notification Service	Уведомления	Отправка Email/Push/Webhook-сообщений	sendEmail, sendPush, webhook	notification_log
Audit Service	Аудит	Сбор и хранение событий аудита	logEvent, queryLog	audit_log

3.2. Диаграмма взаимодействия сервисов

Синхронные взаимодействия (REST через API Gateway):

- Client → API Gateway → User Service (auth, profile)
- Client → API Gateway → Document Service (upload, getVersions)
- Client → API Gateway → Consultation Service (create, history, addMessage)
- Client → API Gateway → Analysis Service (getResult)
- Analysis Service → Legal KB Service (search, getProvision) — внутренний REST

Асинхронные взаимодействия (Apache Kafka):

- Document Service → [DocumentUploaded] → Analysis Service
- Consultation Service → [AnalysisRequested] → Analysis Service
- Analysis Service → [AnalysisCompleted / AnalysisFailed] → Notification Service, Audit Service
- Все сервисы → [AuditEvent] → Audit Service

Рис. 2 — Диаграмма взаимодействия микросервисов LegalAssist AI

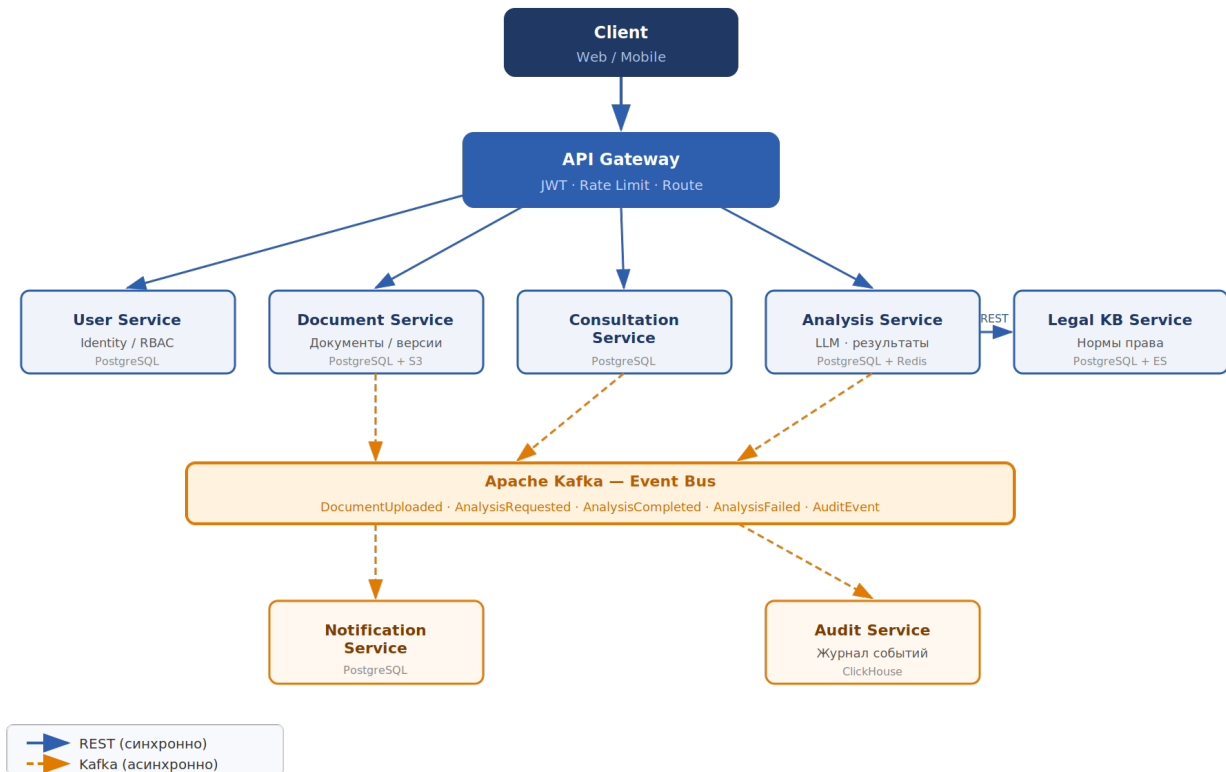


Рис. 2 — Диаграмма взаимодействия микросервисов LegalAssist AI

3.3. Анализ границ сервисов

Обоснование выбора границ:

- Analysis Service выделен отдельно, поскольку LLM-запросы имеют принципиально другой профиль нагрузки (долгие, ресурсоёмкие) и должны масштабироваться независимо от остальных сервисов.
- Legal KB Service отделён от Consultation Service: база знаний обновляется независимо от потока консультаций и может использоваться как самостоятельный поиск (в том числе из Analysis Service).
- Audit Service изолирован как чистый потребитель событий — это гарантирует целостность журнала даже при деградации других сервисов.

Альтернативы и риски:

- Слияние Consultation + Analysis упростило бы разработку, но нарушило бы принцип единственной ответственности и ухудшило бы масштабируемость.
- Избыточная декомпозиция (отдельный сервис на каждую таблицу) привела бы к chatty-интерфейсу и высоким накладным расходам на сеть.

Микрорефлексия. При анализе границ сервисов стало понятно, что главный критерий — независимость масштабирования и независимость изменений, а не просто логическое разделение. Analysis Service — наглядный пример: даже небольшая нагрузка LLM требует отдельного пула ресурсов. Именно анализ NFT из раздела 1 дал основание для этого решения.

4. Архитектура данных и согласованности

4.1. Модель владения данными

Сервис	Набор сущностей	Тип хранилища	Аргументация
User Service	users, roles, user_roles	PostgreSQL	Строгая схема, транзакции, поддержка RBAC
Document Service	documents, document_versions	PostgreSQL + S3/MinIO	Метаданные в SQL; бинарные файлы в объектном хранилище
Consultation Service	consultations, chat_sessions, chat_messages	PostgreSQL	Реляционность, история диалогов, транзакции
Analysis Service	analysis_results, citations	PostgreSQL + Redis	SQL для постоянного хранения; Redis — кэш текущих задач
Legal KB Service	legal_sources, legal_provisions	PostgreSQL + Elasticsearch	Полнотекстовый поиск по нормам через Elasticsearch
Audit Service	audit_log	ClickHouse (append-only)	Аналитика, неизменяемость записей, партиционирование по времени
Notification Service	notification_log	PostgreSQL	Отслеживание статусов доставки сообщений

Каждый сервис владеет только своей схемой БД. Прямой доступ к чужой базе запрещён — взаимодействие исключительно через API или события Kafka.

4.2. Межсервисная согласованность

Ключевой процесс — «Создание и завершение анализа» (Saga Choreography):

- Шаг 1: Consultation Service публикует событие AnalysisRequested в Kafka.
- Шаг 2: Analysis Service создаёт AnalysisJob (статус PENDING), обращается к LLM и Legal KB Service.
- Шаг 3: Analysis Service публикует AnalysisCompleted (статус DONE) или AnalysisFailed (статус FAILED).
- Компенсация: при AnalysisFailed Consultation Service переводит консультацию в статус ERROR; Notification Service уведомляет пользователя об ошибке.

Eventual Consistency: статус консультации обновляется асинхронно при получении AnalysisCompleted. До этого момента клиент видит статус PROCESSING — это приемлемо, так как LLM-анализ занимает от нескольких секунд до нескольких минут.

4.3. Стратегии эволюции и миграции данных

- Версионирование схем БД через Flyway/Liquibase с обратно совместимыми миграциями.
- Принцип expand/contract: добавлять колонки можно сразу, удалять — только после двух релизов (чтобы старый код успел перейти).
- Версионирование событий Kafka: поле schemaVersion в envelope сообщения; консьюмеры игнорируют неизвестные поля (forward compatibility).
- Сценарий миграции из монолита (если применимо): strangler fig — последовательный вынос доменов, начиная с Legal KB Service (минимальные зависимости), заканчивая Consultation Service.

Микрорефлексия. Раздел данных оказался наиболее насыщенным по количеству принятых решений. Выбор хранилищ (PostgreSQL для транзакционных данных, Elasticsearch для поиска, ClickHouse для аудита) продиктован характером нагрузки каждого домена. Впервые осознанно применил паттерн Saga Choreography — стало понятно, почему он предпочтительнее Orchestration: меньше связанности между участниками процесса.

5. Архитектура API и интеграций

5.1. Внешний API системы

API Gateway (Kong / nginx) выполняет роль единой точки входа для всех клиентских запросов:

- Аутентификация: проверка JWT (Bearer, RS256) на уровне шлюза — сервисы не занимаются auth самостоятельно.
- Rate limiting: 60 req/min для неавторизованных пользователей, 300 req/min для авторизованных.
- Версионирование: /api/v1/... и /api/v2/... — параллельная поддержка двух версий в течение шести месяцев после выпуска новой.
- Документация: OpenAPI 3.0, публикуется по /api/docs.

5.2. Межсервисные контракты

Контракт 1 — POST /api/v1/documents (Document Service):

Request: multipart/form-data

file: Binary

title: string

docType: "CONTRACT" | "CLAIM" | "POA"

Response 201:

```
{ "id": UUID, "status": "UPLOADED", "createdAt": ISO8601 }
```

Идемпотентность: заголовок Idempotency-Key (UUID) — повторный запрос возвращает тот же ресурс без повторной обработки.

Контракт 2 — POST /api/v1/consultations/{id}/analyze (Consultation Service):

Request:

```
{ "documentVersionIds": [UUID], "question": string }
```

Response 202:

```
{ "analysisJobId": UUID, "status": "PENDING" }
```

Идемпотентность: повторный вызов с теми же параметрами возвращает существующий job без создания нового.

Контракт 3 — Kafka-событие AnalysisCompleted:

```
{  
  "schemaVersion": 1,  
  "eventType": "AnalysisCompleted",  
  "jobId": UUID,  
  "consultationId": UUID,  
  "riskLevel": "LOW" | "MEDIUM" | "HIGH",  
  "conclusionSnippet": string,  
  "completedAt": ISO8601  
}
```

5.3. Интеграции с внешними системами

- LLM Provider (OpenAI / YandexGPT) — REST API. Anti-corruption layer (ACL) в Analysis Service: адаптер изолирует детали конкретного провайдера; системный код работает через единый интерфейс LLMPort. Смена провайдера не затрагивает бизнес-логику.
- Email/Push Provider (SendGrid, Firebase FCM) — REST API. ACL в Notification Service нормализует форматы разных провайдеров.
- Object Storage (S3 / MinIO) — S3 API. Абстрагировано через интерфейс StoragePort в Document Service.
- Государственные реестры (Консультант+, ГАС «Правосудие») — плановая интеграция через REST с периодической синхронизацией: batch job в Legal KB Service обновляет базу знаний по расписанию.

Микрорефлексия. Проектирование контрактов выявило важность идемпотентности: в распределённой системе клиент всегда должен иметь возможность безопасно повторить запрос без побочных эффектов. Принцип ACL особенно ценен при интеграции с LLM-провайдерами — он позволяет сменить провайдера без изменения бизнес-логики Analysis Service.

6. Качественные атрибуты и кросс-сервисные решения

6.1. Масштабируемость и производительность

- Analysis Service: горизонтальное масштабирование через HPA (по CPU и длине очереди Kafka), до 10 реплик в пиковой нагрузке. Узкое место — вызов LLM API (latency 5–30 с); решение — асинхронная очередь Kafka и polling от клиента.
- Document Service: масштабируется по числу загружаемых файлов; компонент Text Extraction Worker выделен в отдельный Deployment.
- Legal KB Service: Elasticsearch-кластер с шардированием по юрисдикциям; результаты поиска кэшируются в Redis (TTL 1 ч).
- API Gateway: stateless, масштабируется линейно. Остальные сервисы — горизонтальное масштабирование через K8s HPA.

6.2. Надёжность и отказоустойчивость

- Timeouts: все исходящие HTTP-запросы — connect timeout 3 с, read timeout 30 с.
- Retry с экспоненциальным backoff и jitter: 3 попытки при вызовах LLM API.
- Circuit Breaker (Resilience4j): открывается при 50% ошибок за 10 с; при открытом состоянии возвращает fallback-ответ «сервис временно недоступен».
- Dead Letter Queue (DLQ): необработанные сообщения из Kafka-топиков направляются в DLQ для ручного разбора.
- Health checks: эндпоинты /health/liveness и /health/readiness на каждом сервисе; K8s перезапускает упавшие поды автоматически.

6.3. Безопасность

- Аутентификация: OAuth 2.0 / OpenID Connect (Keycloak). JWT (RS256) с TTL 15 мин; refresh token в HTTPOnly-cookie, TTL 7 дней.
- Авторизация: RBAC (роли USER, PREMIUM_USER, ADMIN, ANALYST). Политики проверяются в API Gateway (грубо) и в каждом сервисе (тонко).
- Шифрование: TLS 1.3 для всех каналов; данные в покое шифруются (PostgreSQL TDE / S3 SSE-AES256).
- 152-ФЗ: персональные данные хранятся только в User Service; остальные сервисы оперируют userId (псевдоанонимизация).
- SAST/DAST интегрированы в CI pipeline; penetration testing — раз в квартал.

6.4. Наблюдаемость

- Логирование: структурированные JSON-логи (logrus / zap), сбор через Fluentd → Loki / ELK. Correlation ID передаётся в заголовке X-Request-ID по всей цепочке вызовов.
- Метрики: Prometheus + Grafana. Ключевые метрики — RPS, error rate, P95/P99 latency, queue depth (Kafka), LLM response time.
- Трассировка: OpenTelemetry → Jaeger. Трейс покрывает полный путь от API Gateway до вызова LLM.
- Алерты: PagerDuty; алерты при error rate > 1%, P95 > 500 мс, queue depth > 1 000.

Микрорефлексия. Раздел качественных атрибутов объединил ранее разрозненные решения в единую картину. Ключевым открытием стала связь между Circuit Breaker и DLQ: Circuit Breaker без DLQ бессмысленен — потерянные сообщения всё равно нужно обработать. Наблюдаемость через OpenTelemetry — наиболее интересная для практического применения часть раздела.

7. Инфраструктурная архитектура и CI/CD

7.1. Целевая платформа развёртывания

Целевая платформа — облако (Yandex Cloud или AWS с поддержкой требований 152-ФЗ) с опциональным гибридным режимом (on-premise для хранения особо чувствительных ПДн).

- Контейнеризация: Docker — все сервисы упакованы в образы, публикуются в Container Registry.
- Оркестрация: Kubernetes (Managed K8s в облаке). Выбор обоснован необходимостью автомасштабирования (HPA/VPA), rolling updates и self-healing.
- Service Mesh: Istio — mTLS между сервисами, circuit breaker и traffic management на уровне инфраструктуры.
- Брокер сообщений: Confluent Cloud (managed) или self-hosted Strimzi в K8s.
- Базы данных: Managed PostgreSQL в облаке; Elasticsearch — self-hosted в K8s.
- IaC: Terraform — идемпотентное создание инфраструктуры; все окружения разворачиваются из одного Helm-чарта с разными values.yaml.

7.2. Архитектура окружений

Окружение	Назначение	Изоляция	Данные
dev	Локальная разработка	Docker Compose	Моковые / синтетические
test	Автотесты, интеграционные тесты	Отдельный K8s namespace	Тестовые fixture-данные
staging	Приёмочное тестирование, нагрузочные тесты	Полный K8s-кластер (уменьшенный)	Анонимизированный слепок prod
prod	Продакшн	Отдельный K8s-кластер	Реальные данные (шифрование)

7.3. Общие принципы CI/CD

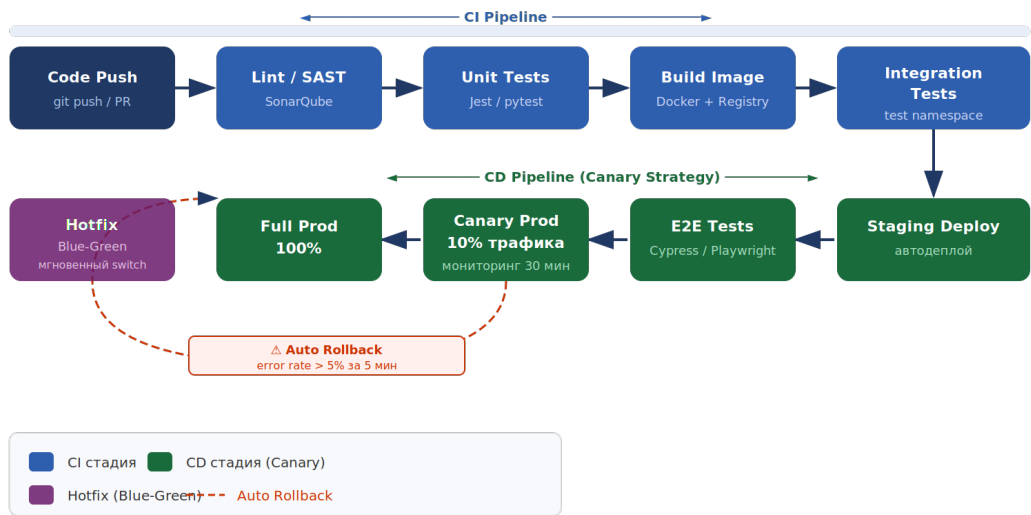
CI Pipeline (GitHub Actions / GitLab CI):

- Lint + SAST (SonarQube) → Unit Tests → Build Docker Image → Push to Registry → Integration Tests (test environment).
- Каждый PR проходит полный pipeline; мерж в main без зелёного статуса заблокирован.

CD Pipeline — стратегия Canary + Blue-Green:

- Feature-ветки → автодеплой в staging после merge в main.
- Prod-релиз: Canary deployment — 10% трафика на новую версию, мониторинг 30 мин (error rate, latency); при норме — постепенное увеличение до 100%.
- Критические hotfix — Blue-Green: быстрое переключение между двумя идентичными средами.
- Автоматический rollback при error rate > 5% в течение 5 минут после деплоя.

Рис. 3 — CI/CD Pipeline LegalAssist AI



Микрорефлексия. Проектирование CI/CD объединило все предыдущие решения. Без наблюдаемости из раздела 6 Canary deployment невозможен — нечем автоматически измерять success rate. Стало очевидно, что автоматический rollback по метрикам — не просто удобная фишка, а обязательный элемент безопасного деплоя в юридической системе, где сбой в продакшне может иметь правовые последствия для клиентов.

Заключение

В ходе работы разработан полный архитектурный проект микросервисной системы LegalAssist AI, охватывающий семь уровней проектирования.

Результаты:

- Выделено восемь микросервисов с обоснованными границами на основе DDD и принципа Single Responsibility.
- Спроектирована модель данных с явным владением (каждый сервис — своя БД) и паттерном согласованности Saga Choreography.
- Определены API-контракты с поддержкой идемпотентности и версионирования.
- Описаны стек наблюдаемости (Prometheus, Jaeger, Loki) и CI/CD с Canary deployment и автоматическим rollback.

Архитектура оправдана для LegalAssist AI разнородными профилями нагрузки (особенно Analysis Service с LLM) и строгими требованиями к безопасности юридических данных. Работа показала, что архитектурные решения нужно принимать комплексно — каждое решение в одном разделе создаёт ограничения или возможности в другом.

Использованные инструменты и промпты для БЯМ:

В работе использовалась модель Claude (Anthropic) для генерации структуры разделов и формулировок.

Промпт 1: «Ты — опытный enterprise-архитектор. Предметная область: LegalAssist AI — автоматизированный юридический помощник. Опиши архитектурные драйверы (НФТ) и их приоритизацию для данной системы. Укажи обоснование приоритетов.»

Промпт 2: «Продолжая архитектурный проект LegalAssist AI, опиши Bounded Contexts в стиле DDD, агрегаты и доменные события для доменов AI Analysis и Document Management. Укажи инварианты агрегатов.»

Промпт 3: «Опиши паттерн Saga Choreography для процесса анализа документа в LegalAssist AI: шаги, компенсирующие транзакции, публикуемые события.»

Подкаст

Ссылка на подкаст:

https://drive.google.com/file/d/12zH_7IRYxRzAwY6yyq9mJL7yZY_FwNT/view?usp=sharing